

FileHandling in Python

File handling refers to the process of performing operations on a file such as creating, opening, reading, writing and closing it, through a programming interface. It involves managing the data flow between the program and the file system on the storage device, ensuring that data is handled safely and efficiently.

Opening a File in Python

To open a file we can use `open()` function, which requires file path and mode as arguments:

```
# Open the file and read its contents
with open('geeks.txt', 'r') as file:
```

This code opens file named geeks.txt.

File Modes in Python

When opening a file, we must specify the mode we want to which specifies what we want to do with the file. Here's a table of the different modes available:

Mode	Description	Behavior
r	Read-only mode.	Opens the file for reading. File must exist; otherwise, it raises an error.
rb	Read-only in binary mode.	Opens the file for reading binary data. File must exist; otherwise, it raises an error.
r+	Read and write mode.	Opens the file for both reading and writing. File must exist; otherwise, it raises an error.
rb+	Read and write in binary mode.	Opens the file for both reading and writing binary data. File must exist; otherwise, it raises an error.
w	Write mode.	Opens the file for writing. Creates a new file or truncates the existing file.
wb	Write in binary mode.	Opens the file for writing binary data. Creates a new file or truncates the existing file.

Mode	Description	Behavior
w+	Write and read mode.	Opens the file for both writing and reading. Creates a new file or truncates the existing file.
wb+	Write and read in binary mode.	Opens the file for both writing and reading binary data. Creates a new file or truncates the existing file.
a	Append mode.	Opens the file for appending data. Creates a new file if it doesn't exist.
ab	Append in binary mode.	Opens the file for appending binary data. Creates a new file if it doesn't exist.
a+	Append and read mode.	Opens the file for appending and reading. Creates a new file if it doesn't exist.
ab+	Append and read in binary mode.	Opens the file for appending and reading binary data. Creates a new file if it doesn't exist.
x	Exclusive creation mode.	Creates a new file. Raises an error if the file already exists.
xb	Exclusive creation in binary mode.	Creates a new binary file. Raises an error if the file already exists.
x+	Exclusive creation with read and write mode.	Creates a new file for reading and writing. Raises an error if the file exists.
xb+	Exclusive creation with read and write in binary mode.	Creates a new binary file for reading and writing. Raises an error if the file exists.

Table of Content

- [Reading a File](#)
- [Writing to a File](#)
- [Closing a File](#)

- [Handling Exceptions When Closing a File](#)

For this article we are using text file with text:

Hello world

GeeksforGeeks

123 456

Reading a File

[Reading a file](#) can be achieved by **file.read()** which reads the entire content of the file. After reading the file we can close the file using **file.close()** which closes the file after reading it, which is necessary to free up system resources.

Example: Reading a File in Read Mode (r)

```
file = open("geeks.txt", "r")
content = file.read()
print(content)
file.close()
```

Output:

```
Hello world
GeeksforGeeks
123 456
```

Reading a File in Binary Mode (rb)

```
file = open("geeks.txt", "rb")
content = file.read()
print(content)
file.close()
```

Output:

```
b'Hello world\r\nGeeksforGeeks\r\n123 456'
```

Writing to a File

[Writing to a file](#) is done using **file.write()** which writes the specified string to the file. If the file exists, its content is erased. If it doesn't exist, a new file is created.

Example: Writing to a File in Write Mode (w)

```
file = open("geeks.txt", "w")
file.write("Hello, World!")
file.close()
```

Writing to a File in Append Mode (a)

It is done using **file.write()** which adds the specified string to the end of the file without erasing its existing content.

Example: For this example, we will use the Python file created in the previous example.

```
# Python code to illustrate append() mode
file = open('geek.txt', 'a')
file.write("This will add this line")
file.close()
```

Closing a File

Closing a file is essential to ensure that all resources used by the file are properly released. `file.close()` method closes the file and ensures that any changes made to the file are saved.

```
file = open("geeks.txt", "r")
# Perform file operations
file.close()
```

Using `with` Statement

`with statement` is used for resource management. It ensures that file is properly closed after its suite finishes, even if an exception is raised. `with open()` as method automatically handles closing the file once the block of code is exited, even if an error occurs. This reduces the risk of file corruption and resource leakage.

```
with open("geeks.txt", "r") as file:
    content = file.read()
    print(content)
```

Output:

```
Hello, World!
Appended text.
```

Handling Exceptions When Closing a File

It's important to handle exceptions to ensure that files are closed properly, even if an error occurs during file operations.

```
try:
    file = open("geeks.txt", "r")
    content = file.read()
    print(content)
finally:
    file.close()
```

Output:

```
Hello, World!  
Appended text.
```

Advantages of File Handling in Python

- **Versatility** : File handling in Python allows us to perform a wide range of operations, such as creating, reading, writing, appending, renaming and deleting files.
- **Flexibility** : File handling in Python is highly flexible, as it allows us to work with different file types (e.g. text files, binary files, CSV files , etc.) and to perform different operations on files (e.g. read, write, append, etc.).
- **User – friendly** : Python provides a user-friendly interface for file handling, making it easy to create, read and manipulate files.
- **Cross-platform** : Python file-handling functions work across different platforms (e.g. Windows, Mac, Linux), allowing for seamless integration and compatibility.

Disadvantages of File Handling in Python

- **Error-prone**: File handling operations in Python can be prone to errors, especially if the code is not carefully written or if there are issues with the file system (e.g. file permissions, file locks, etc.).
- **Security risks** : File handling in Python can also pose security risks, especially if the program accepts user input that can be used to access or modify sensitive files on the system.
- **Complexity** : File handling in Python can be complex, especially when working with more advanced file formats or operations. Careful attention must be paid to the code to ensure that files are handled properly and securely.
- **Performance** : File handling operations in Python can be slower than other programming languages, especially when dealing with large files or performing complex operations.